main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};
struct TreeNode* createNode(int value) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
struct TreeNode* insert(struct TreeNode* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    }
    else if (value > root->data) {
        root->right = insert(root->right, value);
    }

    return root;
}
void display(struct TreeNode* root) {
    if (root != NULL) {
        display(root->left);
        printf("%d ", root->data);
        display(root->right);
```

Output

Clear

```
Tree created! (Displayed in sorted order): 20 30 40 50 70


=== Code Execution Successful ===
```

main.c

Share Run

```c
        return createNode(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    }
    else if (value > root->data) {
        root->right = insert(root->right, value);
    }

    return root;
}
void display(struct TreeNode* root) {
    if (root != NULL) {
        display(root->left);
        printf("%d ", root->data);
        display(root->right);
    }
}

int main() {
    struct TreeNode* root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);

    printf("Tree created! (Displayed in sorted order): ");
    display(root);
    printf("\n");

    return 0;
}
```

Output

Clear

```
Tree created! (Displayed in sorted order): 20 30 40 50 70


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdlib.h>
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};
struct TreeNode* createNode(int value) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode
        ));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
void pre_order_traversal(struct TreeNode* root) {
    if (root == NULL) {
        return;
    }
    printf("%d ", root->data);
    pre_order_traversal(root->left);
    pre_order_traversal(root->right);
}

int main() {
    struct TreeNode* root = createNode(10);
    root->left = createNode(20);
    root->right = createNode(30);
    root->left->left = createNode(40);
    root->left->right = createNode(50);

    printf("Pre-order Traversal (Root -> Left -> Right): \n");
    pre_order_traversal(root);
    printf("\n");
```

Output

Clear

```
Pre-order Traversal (Root -> Left -> Right):
10 20 40 50 30


=== Code Execution Successful ===
```

Ashwin
Brand B
SPONSORE